2026—2027 学年第一学期

《机器人感知与控制综合实践》报告

第 11 组

作品名称

基于 ROS 2 的差速机器人

PID 左墙追踪与 PP 迷宫搜索

课程代码：(2026-2027-1)-S0610350-02

学生 1（组长）：丁俊豪　学号：24061435

学生 2（组员）：刘韩旭　学号：24061412

提交日期：2026 年 11 月 3 日

目 录

1 需求分析.....3

2 系统总体方案.....4

3 环境设计与模型实现.....6

4 算法原理设计.....9

5 测试结果与现象分析.....21

6 问题分析、解决方法与改进.....27

7 分工、AI 使用说明与开源信息.....29

8 总结、展望与课程建议.....31

9 参考文献.....32

10 其他附件说明.....33

1 需求分析

1.1 项目背景与目标

地老鼠走迷宫实验要求差速机器人从未知环境入口出发，利用传感器发现通道、识别墙体并自主驶向出口。本组围绕同一 TurtleBot3 Burger 仿真底盘完成两个独立项目：PID 左墙追踪用于连续走廊中的局部反应控制；PP 迷宫搜索用于含岔路和死胡同的在线道路探索。

基础部分覆盖 ROS 2 通信、机器人模型、激光分区、差速运动和左墙跟随；拓展部分采用 Pure Pursuit 连续跟踪，维护道路拓扑，利用在线地图校正和已知死路判定，出迷宫后以 A* 重新规划返航路线。PID 基于局部激光闭环控制，PP 在行驶中同步构建拓扑图与 SLAM 地图，自主生成探索路线。[1][2]

1.2 输入输出与功能需求

表 1 需求与工程实现对应

类别	具体要求	对应实现
感知输入	解析 360 点平面激光；获取位姿、速度和时间戳	/scan、/odom、/imu、TF
PID 任务	保持接近通道半宽的左墙距离；弯道不断线；出口停稳	maze_wall_pid / left2_lidar
PP 任务	左右路口均登记、左优先选择、死路回溯、出迷宫再返航	maze_bringup / polyline_follower.py
执行输出	发布线速度 v、角速度ω；丢数据或任务结束持续零指令	/cmd_vel → /burger/cmd_vel
可复现性	一键 launch；等待 start；参数独立；可视化路径和状态	merged.launch.py、maze2.launch.py

实验从接口、运动行为和任务完成情况三个层面评价系统：检查雷达与 TF 有效性，观察沿墙追踪、岔路选择与死路回溯，记录通过时间、返航误差及结束后的停车状态。

2 系统总体方案

2.1 方案划分与总体架构

系统按感知、决策、运动控制三层组织。感知层提供有效激光点、融合里程计与在线 SLAM 信息；PID 决策层管理普通追踪、寻墙恢复和出口停车，PP 决策层以拓扑图管理支路探索，以 SLAM 修正拓扑几何、辅助判定死路。需要转移到其他探索节点时，在已知拓扑图上运行 Dijkstra；离开迷宫后，在 SLAM 栅格图上运行 A*返航。控制层用墙线误差控制或 PP 跟踪执行参考路线，经统一速度保护驱动差速插件。两个项目分别启动。

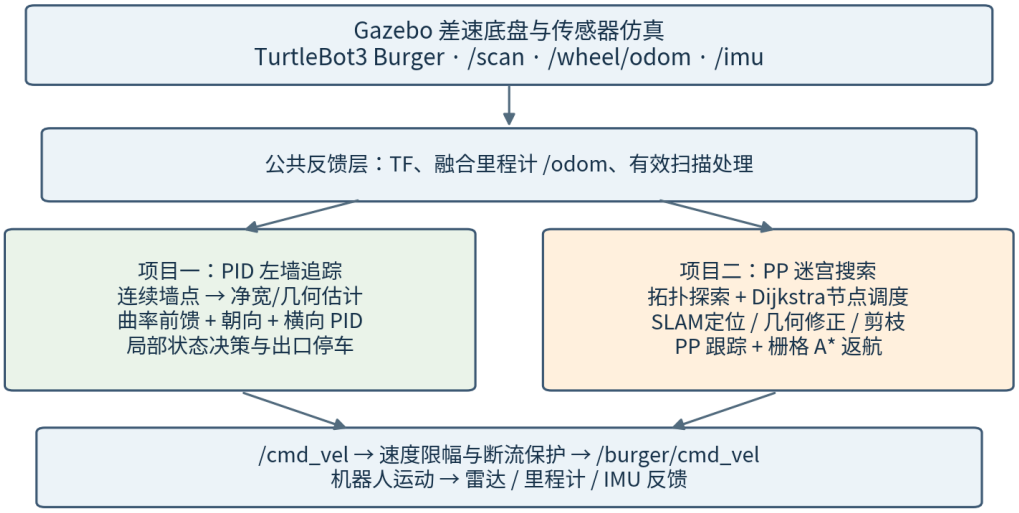


图 1 系统总体架构

2.2 两项目的功能特点

表 2 两项目方案比较

项目	PID 左墙追踪	PP 迷宫搜索
主要环境	合并的直道、L/U/Z/S 弯走廊	maze2 多岔路矩形迷宫
环境记忆	当前墙点与最近有效净宽	拓扑图记录支路与连接； SLAM 提供定位及几何辅助
控制对象	距离与墙线方向误差	参考折线上的前瞻目标点
路径决策	局部沿墙、障碍恢复	左优先选支路、拓扑 Dijkstra 转移、栅格 A*返航

项目	PID 左墙追踪	PP 迷宫搜索
结束状态	EXIT_REACHED：驶出并停稳	HOME_REACHED：返航到起点并停止

选择差速模型的原因是硬件结构清晰，输入为前进速度和偏航角速度，便于将几何轨迹转换为轮速。激光雷达提供墙面和通道观测，里程计提供位姿与速度反馈，IMU 辅助获得实际偏航响应。

2.3 工程模块与可行性

工程由 maze_wall_pid 与 maze_bringup 两个软件包组成，各自包含 launch、仿真准备、里程计和速度保护模块；共享的官方模型依赖位于 src/vendor。算法、参数、world 与 RViz 独立组织，支持分别调参、切换场景与维护可视化配置。两个项目通过统一的传感器和速度接口连接仿真底盘。

3 环境设计与模型实现

3.1 软件环境与机器人模型

工程运行于 Ubuntu 22.04、ROS 2 Humble、Gazebo Classic 11；采用 ROBOTIS 官方 Humble 分支的 TurtleBot3 描述与仿真资源，模型来源及许可证保存在 vendor 包内。

表 3 模型元素及实际配置

标签/组件	作用	本项目中的使用
link / joint	描述刚体与关节连接、运动类型	底盘与传感器固定连接；左右轮为连续转动关节
visual	显示几何、网格、颜色	RViz 与 Gazebo 的 Burger 外观
collision	参与物理接触判断的几何	车体、车轮、墙体碰撞
inertial	质量及惯性张量	仿真运动响应和停止过程
差速尺寸	轮半径 0.033 m、轮距 0.160 m	由模型与插件配置读取
激光安装	相对 base_link 为(-0.032,0,0.172) m	先经 TF 变换到 base_footprint 再处理

base_link 相对 base_footprint 的固定高度为 0.010 m，因此激光坐标原点相对 base_footprint 约为 0.182 m；其世界坐标高度由固定变换、出生位置和落地状态共同确定。RViz 显示机器人模型和消息，Gazebo 根据碰撞、重力、摩擦及惯性推进动力学。

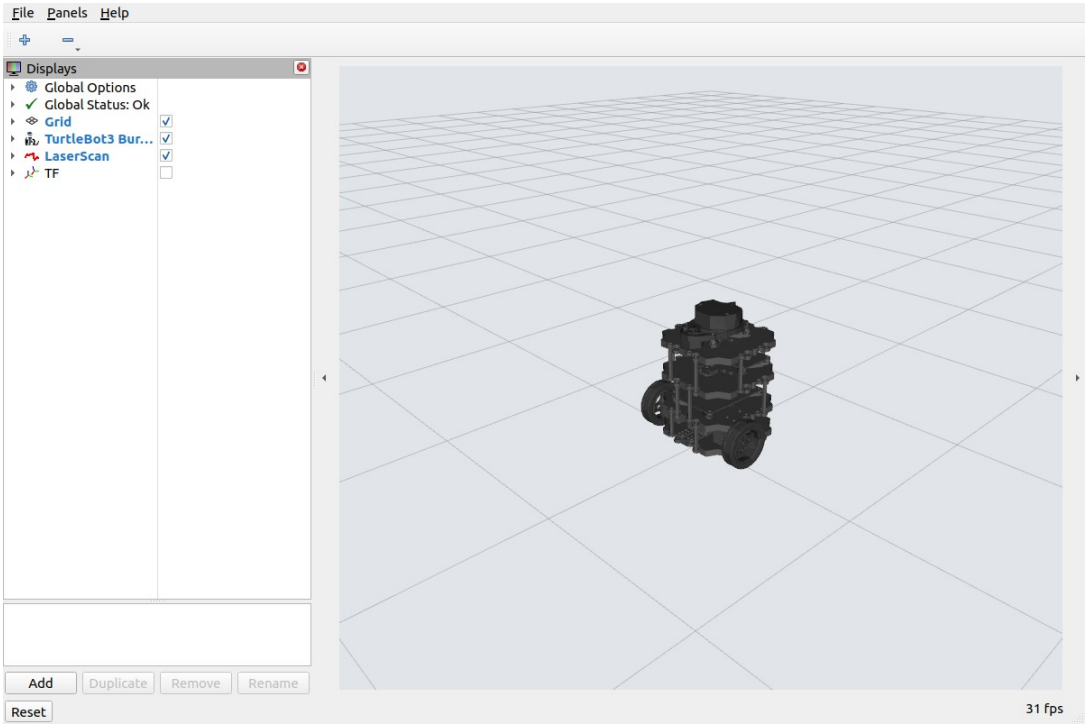


图 2 TurtleBot3 Burger 模型与雷达的 RViz 显示

3.2 两张 world 与出生位置

PP 使用 maze2.world：矩形墙由 box 几何构成，常见墙厚 0.050 m、墙高 0.300 m，底部开口进入。PID 使用 merged_maze.world：用矩形墙段连接多种弯道，典型墙厚 0.300 m、墙高 1.200 m、名义通道净宽 1.700 m。内弯由重叠墙段形成局部锯齿，控制器通过墙点平滑与曲线拟合提取连续参考线。所有墙段同时配置 visual 和 collision，分别用于显示与物理接触。

表 4 当前 launch 默认出生位置

项目	出生位置(x,y,z) / m	yaw / rad	说明
PP / maze2	(-2.275, -3.800, 0.010)	$\frac{\pi}{2}$	由下方缺口朝+Y 进入
PID / merged	(-2.450, 0.6701, 0.010)	0	中央直道左端内侧 0.20 m，朝+X 进入

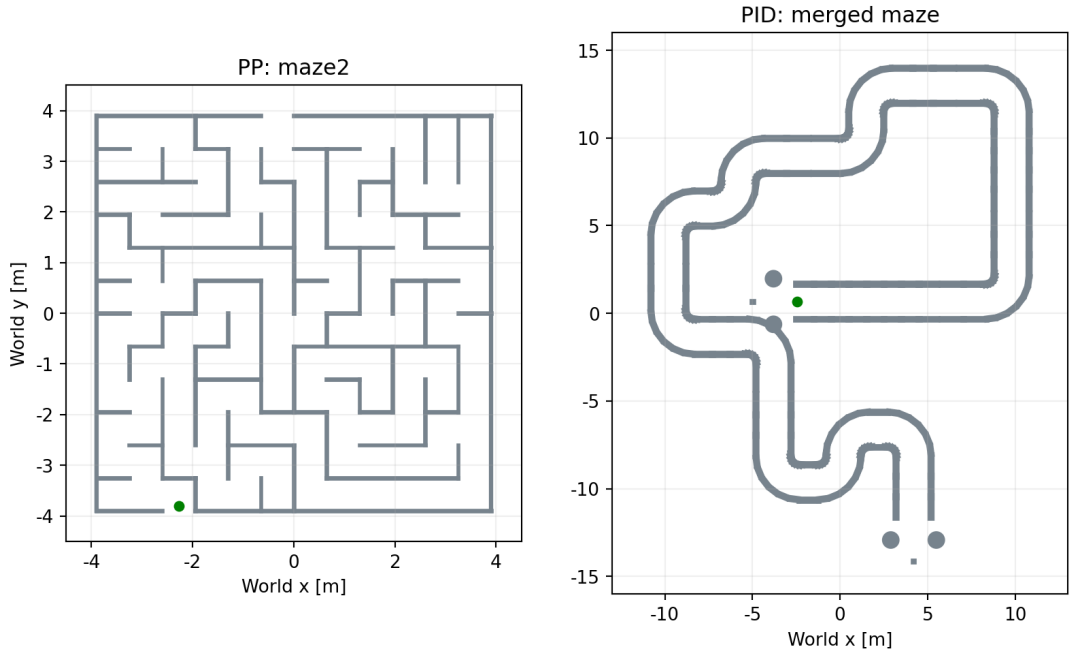


图 3 两张 world 的碰撞几何俯视布局，绿点为出生位置

合并图入口两侧墙内表面为$y=-0.1799$和$y=1.5201$，故中心$y=0.6701$、左右初始距离均约 0.85 m。入口绿色标记本身有碰撞体，车辆不生成在标记中心。导入图中的 ambient 类型光源不受当前 SDF 支持，工作副本改用 scene ambient，墙体几何保持原样。

3.3 一键启动、构建与可视化

构建及启动命令：依据当前两包 README；启动后在同一终端输入 start。

```
cd ~/maze_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install --packages-up-to maze_wall_pid maze_bringup
source install/local_setup.bash
# 每次只选择一个项目
ros2 launch maze_wall_pid merged.launch.py
# 或: ros2 launch maze_bringup maze2.launch.py
```

launch 先加载内部 launch_support/simulation.py，准备 Gazebo 世界与临时车模；机器人描述由 robot_state_publisher 发布，spawn_entity 成功后解除物理暂停，再启动图形客户端。算法节点和 RViz 由同一入口组织，WAIT_START 阶段持续零指令。PID 默认不启动 SLAM；PP 默认启动 slam_toolbox 在线建图。模型消息、TF 和 SLAM 使用仿真时间，控制指令定时发布与数据超时检测采用稳态计时。

表 5 与当前仓库一致的核心文件定位

层次	主要文件	作用
PID 入口	maze_wall_pid/launch/merged.launch.py	加载合并 world、参数、控制器和显示
PID 核心	src/left2_lidar.cpp；include/.../left2_trace.hpp	扫描、几何拟合、有界 PID、状态机
PP 入口	maze_bringup/launch/maze2.launch.py	加载 maze2、在线 SLAM 及 PP 探索节点
PP 核心	scripts/polyline_core.py；polyline_follower.py	拓扑、PP 与任务阶段组织
地图与返航	scripts/branch_map.py；return_planner.py	地图约束、死路证明与 A*
公共执行	scripts/maze_odometry.py；velocity_guard.py	轮里程计/IMU 融合、限幅和断流停车

RViz 中 PID 固定坐标系用 odom，PP 在线建图时用 map；共同显示 RobotModel、LaserScan、路径与 MarkerArray。TF 链包括 map→odom→base_footprint→base_link→base_scan，其中 map→odom 仅在建图方案中存在。视角 Target Frame 设为固定坐标系，并保留 MoveCamera 等工具，使视图既不随车漂移，也能手动平移和缩放。

4 算法原理设计

4.1 差速运动与比例闭环基础

差速底盘通过左右轮速度差产生转向，以前进速度和偏航角速度控制平面运动。设车体位置为(x,y)、偏航角为θ、前进速度为 v、偏航角速度为ω，左右轮角速度分别为Ω_L、Ω_R，轮半径 r、轮距 b，则运动学关系为：

$$\frac{dx}{dt}=v\cos\theta,\frac{dy}{dt}=v\sin\theta,\frac{d\theta}{dt}=\omega$$

$$v=\frac{r}{2}(\Omega_R+\Omega_L),\omega=\frac{r}{b}(\Omega_R-\Omega_L)$$

开环直行取$\omega=0$、v 为常值；原地转向取$v=0$。点到点比例闭环由里程计计算距离误差e_ρ和目标方位误差e_θ，再通过比例增益与限幅生成速度，基本关系为：

$$e_\rho=\sqrt{(x_g-x)^2+(y_g-y)^2}$$

$$e_\theta=wrap\left[atan2(y_g-y,x_g-x)-\theta\right]$$

$$v=sat(k_\rho e_\rho,0,v_{max}),\omega=sat(k_\theta e_\theta,-\omega_{max},\omega_{max})$$

当前 PP 节点中，track_reference 的大角度原地对准采用$\omega=sat(1.8\alpha)$，达到允许朝向误差后恢复前进；返航接近起点时采用$v\le 1.2\,d_{home}$的比例限速，并在 0.10 m 容差内结束。路径由多个目标段组成，前一段完成后按单调进度切换到下一段，避免反复选择已通过的航点。

4.2 激光解析、分区与有效性处理

LaserScan 给出起始角、角分辨率、量程和 ranges 数组。第 i 束角度为$\alpha_i=\alpha_{min}+i\,\Delta\alpha$，对应平面点为$(r_i\cos\alpha_i,r_i\sin\alpha_i)$，再经 TF 变换到控制坐标系。消息以+X 为零角、绕+Z 逆时针为正方向，分区读取依据消息中的角度参数换算。

表 6 激光分区与用途

用途	角度区域	读取/判定
PID 前方	0°±15°	有效距离最小值，小于 0.50 m 进入右转恢复
PID 左墙	60°～90°	距离监测；正常控制使用连续墙点几何

用途	角度区域	读取/判定
PID 右侧	-90°±15°	监测及双墙几何辅助
PID 出口侧向	正左/正右各±8°	与前方联合判定开阔，避免宽扇区扫到远处柱
PP 通行判断	按候选点方位查询邻近射线	量程外返回未知；路口连续观测后登记

角度差用$atan2(\sin\Delta\alpha,\cos\Delta\alpha)$归一化，兼容 0～2π 和-π～π 布局。NaN 和越界量程作为无效观测处理；正无穷在扇区“无回波”判断中按最大量程处理，墙面拟合使用有限的有效回波点。PID 扇区要求至少 3 束且 80%以上样本有效；TF 缺失、扫描或里程计超时时停车，出口确认按不同时间戳的有效扫描累计。

4.3 PID 左墙追踪与双墙几何辅助

本项目将左墙视为参考线，利用连续激光墙点进行单边沿墙追踪，在 base_footprint 坐标系中以米为单位计算误差。当前合并图采用曲率前馈、朝向修正和横向误差 PID 共同控制角速度，使机器人沿参考线前进；几何拟合失效时采用加权参考线误差作为反馈。

4.3.1 连续墙点与参考线提取

trace_edge 先按射线方位排序，在车体左侧近处选种子点，沿递减射线角从近墙向前跟踪。相邻点间距超过 0.10 m、越出局部范围或达到追踪长度时终止；每条有效链至少 7 点、长度至少 0.12 m，避免把真实缺口两侧的墙强行接成一条线。原始链经最多 5 点滑动平均，再按 0.015 m 弧长等距采样，使近处密集雷达点不因数量多而获得过高权重。

在等弧长墙线上，以 0.18 m 半窗口拟合参数曲线$x(s),y(s)$，由切向求指向通道内侧的右法向，偏移距离 d 生成目标参考线；偏移后的参考线再次按 0.015 m 采样，并取前 0.85 m 用于加权误差计算。这样能够处理直墙、垂直墙面及连续弯道。

$$p_{target}(s)=p_{left}(s)+d\,n_{right}(s),d=\frac{W_{corridor}}{2}$$

4.3.2 净宽估计与双墙可靠性融合

update_corridor 在车体近侧墙点上进行带离群点剔除的直线拟合，净宽取左右墙面的垂直距离之和。合并图仅在两侧拟合均有效、残差各小于 0.012 m、墙面朝向均接近车身且彼此夹角差小于 0.15 rad 时更新宽度，范围限定为 1.0～2.6 m。估计量经 0.60 s 一阶滤波和 0.40 m/s 变化率限制，d 取其一半，初值为 0.85 m；缺口、遮挡或弯道中保留最近可靠宽度。

当前正常控制启用 wall_geometry：直接拟合左墙局部前 0.80 m 弧段的二次参数曲线，在法向偏移线上搜索距车轴最近的参考点，得到横向误差e_y、切向朝向误差e_ψ和曲率κ。横向误差取该参考点在局部左法向上的有符号距离，朝向误差取切向相对车头的角度；参数化拟合避免了$y(x)$在近垂直墙面处失效。

$$e_y=\frac{-t_y p_x+t_x p_y}{\|t\|},e_\psi=atan\,2(t_y,t_x)$$

$$\kappa_{target}=\frac{\kappa_{wall}}{1+d\,\kappa_{wall}}$$

右侧点云先镜像为左侧墙线进入同一计算流程，再将横向、朝向和曲率符号翻转回车体坐标。corridor_geometry 按$q=\frac{L^2}{0.01+RMS}$评价两侧质量，其中 L 最多取 0.80 m；两侧估计接近时按 q 归一化融合，朝向差超过 0.35 rad 或横向差超过 0.25 m 时选质量更高的一侧。一侧无效时使用另一侧，因此 S 弯内侧短墙被遮挡时仍可利用清晰的外侧墙。

4.3.3 复合角速度、离散 PID 及回退误差

几何有效时，曲率先限幅至±1.2 m^{-1} 并经 0.20 s 滤波，随后计算曲率前馈$v\kappa$、朝向修正$3.0\,v\,e_\psi$及横向误差 PID。左侧墙链存在而几何无效时，采用 error_calculater 计算参考线各点横向坐标的近端加权误差，仅用该误差驱动 PID。右侧有效几何也计为找到墙线；两种控制方式切换时重置 PID，避免历史误差带入新模式。

$$\omega_{des}=v\,\kappa_{filtered}+3.0\,v\,e_\psi+PID(e_y)$$

$$e_{trace}=\frac{1}{N}\sum_{i=0}^{N-1}w_i\,y_i$$

$$w_i=\begin{cases}1-\dfrac{i}{100}, & 0\le i\le 30,\\ 0.7\exp[-0.064(i-30)], & i>30.\end{cases}$$

横向误差 PID 增益$K_p/K_i/K_d$为 4/0/12，当前实际采用 PD 项；积分累计仍设±0.15 限幅。D 项取相邻有效帧误差差分并限制在±0.10，不除以采样周期；回退误差按点数 N 归一化。扫描回调计算新控制量，50 ms 定时器持续发布最近指令并检测传感器超时。

源码：maze_wall_pid/include/maze_wall_pid/left2_trace.hpp，PID::update。

```
const double diff=initialized ? std::clamp(error-previous,-.10,.10) : 0.;
p_term=p*error; i_term=i*integral; d_term=d*diff;
previous=error; initialized=true;
return std::clamp(p_term+i_term+d_term,-limit,limit);
```

正常追踪再按$\omega_{des}+0.30(\omega_{des}-\omega_{measured})$修正偏航响应，限幅至±2.0 rad/s，并按 4.0 rad/s² 限制相邻帧指令变化。普通追踪线速度设为 1.5 m/s，角速度反馈来自融合里程计；前方危险和寻墙恢复使用独立状态指令。

4.3.4 控制优先级与状态切换

控制顺序为：完成锁定、等待启动/暂停、数据有效性、出口状态、前方危险、连续丢线、正常追踪、短时缺线。扫描或里程计超过 0.7 s、TF 缺失或扫描无效时发布零指令并重置控制历史。出口状态接管运动后，按第 4.6 节完成确认、驶出、制动和停稳。

普通沿墙状态下，正前方小于 0.50 m 时令$v=0$、$\omega=-0.60$ rad/s；小于 0.25 m 时标记 CLEARANCE_RECOVERY，右转指令保持相同。有效墙线连续丢失至少 2 帧时，以设定线速度和+0.60 rad/s 左转寻墙；仅短缺 1 帧时保持上一角速度。前方危险优先于丢左线寻墙，支持 S 形走廊中的右转恢复。PID 项目主要使用局部激光闭环，里程计用于偏航反馈、累计行驶距离及出口停稳判断，默认不启动 SLAM。

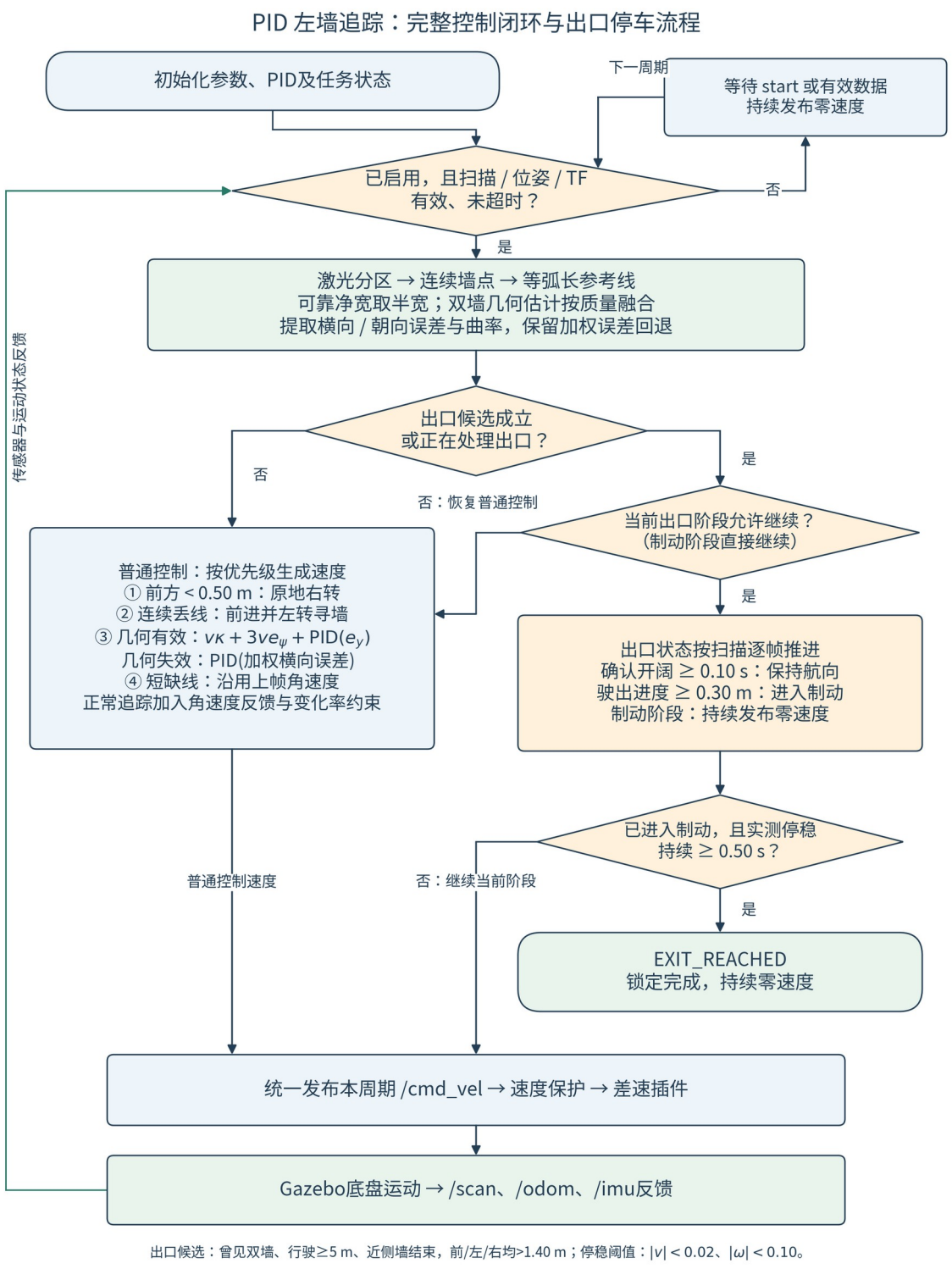


图 4 PID 左墙追踪闭环与出口停车状态流程

4.4 PP 几何追踪与前瞻点选择

Pure Pursuit 将拓扑探索或返航规划生成的参考路径转换为连续速度指令。将前瞻点转换到车体坐标得到(x_L, y_L)，设$L^2 = x_L^2 + y_L^2$，由通过当前位置、切向与车头方向一致的圆弧可得$\kappa = \frac{2\,y_L}{L^2}$，故$\omega = v\kappa$。[1]

$$x_L = \cos\theta\,\Delta x + \sin\theta\,\Delta y,\ y_L = -\sin\theta\,\Delta x + \cos\theta\,\Delta y$$

$$\kappa = \frac{2\,y_L}{x_L^2 + y_L^2},\ \omega = v\,\kappa$$

源码：maze_bringup/scripts/polyline_core.py，pure_pursuit。

```
def pure_pursuit(position,yaw,target,speed):
    delta=np.asarray(target)-position;c,s=math.cos(yaw),math.sin(yaw)
    x=c*delta[0]+s*delta[1];y=-s*delta[0]+c*delta[1]
    return speed,2*speed*y/max(x*x+y*y,1e-6),math.atan2(y,x)
```

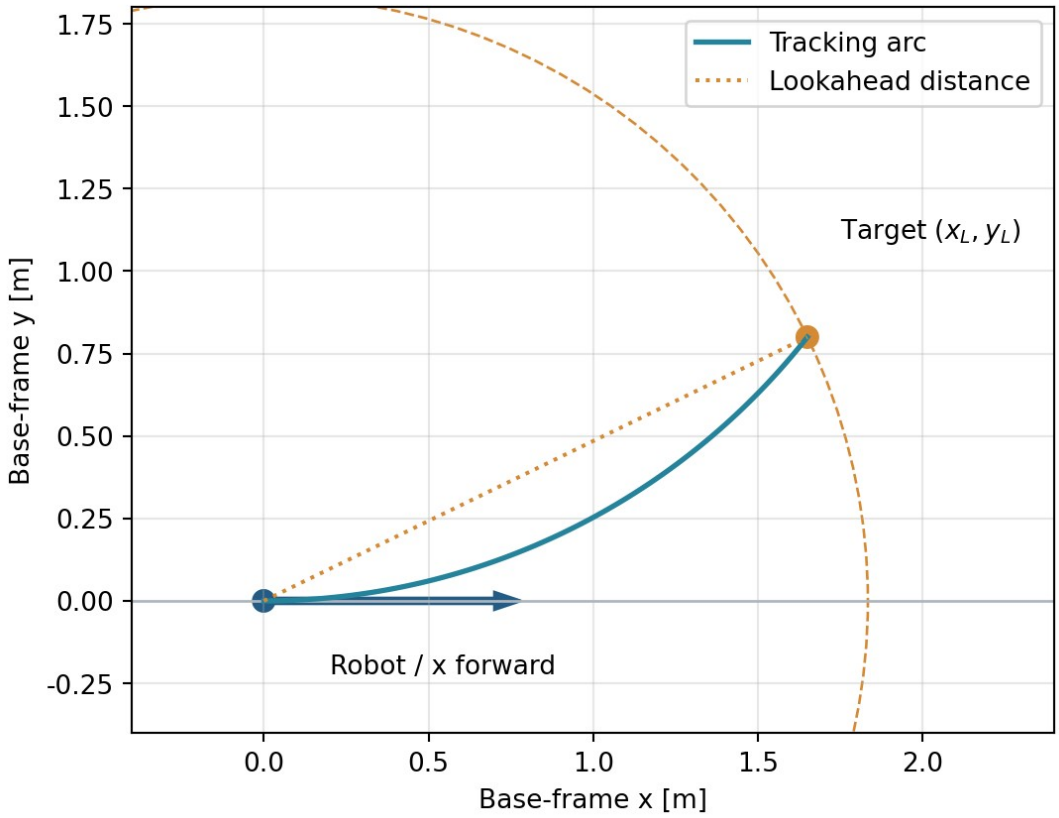


图 5 Pure Pursuit 几何关系示意

前瞻点采用参考折线与前瞻圆的前向交点，当前前瞻半径 0.32 m。若仅沿折线累计固定弧长，直角附近的实际欧氏前瞻距离会缩短，最近投影换段时还可能跳点；圆交点使转弯几何更连续。控制前用实测速度预测 0.10 s 位姿，再计算前瞻目标，并采用

角速度反馈抑制底盘响应滞后。普通 PP 巡航上限为 0.53 m/s，角速度上限 4.3 rad/s；大角度对准上限 2.4 rad/s，朝向误差进入 0.50 rad 容差后可恢复前进。

4.5 拓扑图与 SLAM 协同建图及 Dijkstra 节点调度

4.5.1 两种地图的职责与信息交互

PP 项目同时维护道路拓扑图和 SLAM 度量地图。拓扑图是探索决策的主体，记录“有哪些岔路、彼此如何连接、哪些方向已选择探索、下一步去哪个节点”；SLAM 提供 map 坐标定位、关键帧位置及占据栅格，辅助校正拓扑几何和识别已完全观测的死路。PP 控制器跟踪由决策层生成的连续参考折线。[2]

表 7 PP 中拓扑图与 SLAM 地图的分工

表示/模块	保存的信息	主要用途
拓扑图 Explorer	Junction 节点、Leg 道路、方向端口及邻接连接	记录支路探索状态，左优先选路，Dijkstra 调度探索节点
SLAM 定位与位姿图	map→odom 变换；稳定 ID 的关键帧位置	统一当前扫描与位姿，随回环修正历史节点、起点和轨迹
SLAM 占据栅格 /map	已知空闲、墙体和未知单元	墙约束校正拓扑、死路剪枝；出口后提供 A*返航地图

PolylineFollower 订阅/scan、/odom、/map 及/slam_toolbox/graph_visualization。update_scan 先用扫描时间戳取得 odom 下雷达位姿，再对该扫描和当前里程计施加同一份最新 map→odom 修正，使两者处于一致地图坐标，避免回环前后变换混用引起参考线突跳。

4.5.2 拓扑节点、支路状态与在线折线生成

Junction 保存位置 point、四方向状态 ports、邻接表 neighbors 和方向到道路 ID 的 branches；Leg 保存起点、终点、方向、目标节点及 visited、retired、map_dead_end 等状态。ports 区分 unknown、blocked、open、tried。选择支路时 visited/tried 置位，表示该道路已纳入当前探索；尚未驶入的预选支路若取消，则恢复为 open。已知死路用 map_dead_end 单独标记，与道路访问状态分别保存。

初始参考段沿车头方向延伸，到实测墙前 0.30 m 结束；未看到封闭端点时先保留可见范围内的临时段，随前进延长。侧向通行参考点由可见墙面偏移确

定。record_side_openings 分别检测左右缺口，候选位置连续 2 帧相差小于 0.08 m 才登记；中途缺口通过 split_current_road 把既有边拆成前后两段，插入共用节点并保存侧支路。

同一节点内用 0.22 m 合并半径、已有方向端口和道路 ID 复用历史连接，反向通过时沿用同一道路。observe_branches 一次保存当前可见的所有方向，再按左、前、右、后选择未尝试且未剪枝的支路；右侧支路可暂缓进入，但保留到后续调度。仍为 unknown 的方向继续观测，暂不作死路处理。

源码：Explorer.select_branch；方向偏移(1,0,-1,2)为左、前、右、后。

```
def select_branch(self, node_id, incoming):
        node=self.nodes[node_id]
        for offset in (1,0,-1,2):
            index=(incoming+offset)%4
            if index not in self.untried(node_id):continue
            edge=self.outgoing(node_id,index)
            edge.visited=True;self.edges[edge.edge_id].visited=True
            node.ports[index]='tried';self.turns+=int(index!=incoming)
            return edge
        return None
```

4.5.3 Dijkstra 如何选择目标节点及最短路线

当前节点没有可探索支路、确认前方死路，或需要回到其他待探索岔路时，frontier_route 从所在拓扑节点执行 Dijkstra。搜索只扩展 neighbors 中已建立连接且 visited 为真的道路，边权取两个端点当前 map 坐标的欧氏距离，可利用已知环路和较短连接完成跨节点转移。

$$w(u,v)=\left\|p_v-p_u\right\|_2, D(v)\leftarrow min\{D(v),D(u)+w(u,v)\}$$

实现用 heapq 按累计距离从小到大取出节点，best 保存当前最短代价。第一个被取出且仍有 untried 支路的非起始节点，就是距离最近的探索目标；untried 同时排除已选择、废弃、地图已证明封闭的支路，以及完全位于已证实死路区域内的节点。因此目标选择与最短路搜索在同一次 Dijkstra 中完成。

源码节选：polyline_core.py，Explorer.frontier_route；累计已知拓扑边长选择目标节点。

```
if node_id!=start and self.untried(node_id):return node_id,path
for heading,target in self.nodes[node_id].neighbors.items():
    edge_id=self.nodes[node_id].branches[heading];edge=self.edges[edge_id]
    if not edge.visited:continue
    length=float(np.linalg.norm(self.nodes[target].point-
self.nodes[node_id].point))
    distance=cost+length
```

start_return 把最短路线保存为连续 Leg 序列：第一段设为当前 leg，其余保存在 return_route，并预取 next_leg。path 将当前段和后续道路端点拼为同一条参考折线，PP 沿折线连续追踪，advance 按行驶进度切换道路。局部新支路按左优先决定；跨节点转移按 Dijkstra 最短距离决定；两者分别负责“探索哪里”和“怎样到达”。

确认死路后可以从当前行驶位置回溯，无需追到无信息增益的最远端点。回溯沿用原节点与边；途中发现新的左侧未探索分支时，登记该节点并重新安排后续道路，取消尚未执行的旧预选支路，避免漏路和重复画平行线。

4.5.4 SLAM 如何辅助修正拓扑几何

位姿图锚定：PoseGraphAnchors.attach 将节点、未连接道路端点等绑定到稳定关键帧 ID。若能找到附近第二关键帧，以两帧连线建立局部方向并保存局部坐标；update_pose_graph 在关键帧优化后恢复地图位置，同步已连接边的终点和当前/待执行路线。on_slam_graph 同时修正记录起点、历史轨迹和返航参考点，保留拓扑节点 ID、连接关系与探索状态。

墙面约束：on_map 取占据值≥65 的墙栅格中心，align_to_walls 在已访问道路两侧分段提取近侧墙面并拟合直线，形成距墙 0.30 m 的法向约束。共享路口联合求解所有相邻道路约束，并加入原位置弱先验，以保持走廊方向上尚无约束的坐标。每次修正最多 0.04 m，且新位置必须落在已知空闲单元；道路端点和待跟踪路线随后同步更新。

4.5.5 SLAM 已知死路判定与提前回溯

BranchMap.check 在支路口向内 0.38 m 设置虚拟封口，两端必须接到实测侧墙；在口内自由单元作八邻域泛洪。只有该连通区域与父走廊不再连接、不触及地图边界、也不邻接任何未知区域，才证明它是仅有一个入口的已观测封闭口袋。判断使用原始自由栅格，不作障碍膨胀，避免抹掉真实窄出口。

未访问支路被证明封闭后从 untried 候选中排除，但原拓扑边及访问记录保留。地图更新时重新建立判定并刷新缓存，几何修正或新开口可撤回旧结论。return_from_mapped_dead_end 还检查正在前进的道路；必要时从车辆当前投影附

近重新证明前方封闭区域，随后立即接入拓扑回溯路线，从而同时处理侧支路死路和正前方死路。

4.6 出口识别与 A* 返航

PID 任务在识别过两侧墙体、行驶至少 5 m 后，要求有效完整扫描、近侧墙结束、前方与正左/正右小扇区超过 1.40 m。合并图连续确认 0.10 s 后保持航向驶出，以 0.30 m 为开始制动的最小进度；随后实测¿v∨¿0.02 m/s、¿ω∨¿0.10 rad/s 持续 0.50 s，锁定 EXIT_REACHED。

出口状态跨帧保持为 VERIFY_OPEN→LEAVING→BRAKING→DONE。前两阶段保存出口处航向，以 2.0 倍朝向误差修正并限幅至±0.50 rad/s；确认阶段开阔条件消失，或驶出阶段重新观测到有效双墙/前方危险时，返回普通沿墙控制。制动阶段持续零指令，实测速度重新超阈值则重置停稳计时；达到确认时间后锁定完成。

PP 使用独立的出口检测器：曾经处于走廊且行驶达到门槛后，前后左右至少三侧开阔，并满足连续开放角与 0.8 s 确认条件。记录出口位置后转入 PLAN_RETURN，在在线已知自由地图上规划回到初始位置的路线。

A*将占据栅格按机器人净空要求膨胀，未知或被占用单元不通行。采用八邻域，禁止对角穿越墙角；实际代价含距离和靠墙惩罚，启发项采用八邻域距离。规划后删除共线点，检查连接安全性，再由 PP 跟踪返航路径。优化目标是在已建地图中兼顾行驶距离和墙面净空。

出口外可能超出 SLAM 栅格裁切边界，ReturnGrid.observe 会在独立返航规划层中加入当前雷达 1.5 m 以内的实测射线：只把射线实际穿过的未知单元补为空闲，保留原墙体并标记实测障碍。该局部补观测不改变探索拓扑和原 SLAM 地图，使出口位置能够接入已有安全道路。

$$f(n)=g(n)+h(n)$$

$$h(n)=max(\Delta r,\Delta c)+(\sqrt{2}-1)min(\Delta r,\Delta c)$$

返航以 SLAM 地图中的记录起点为目标，距离小于 0.10 m 时进入 HOME_REACHED 并持续停车。暂时没有安全路径时保持停车，等待地图更新后重新规划。

4.7 关键参数

表 8 2026 年 11 月 3 日当前源码参数

参数	PID 合并图	PP maze2
巡航速度	1.5 m/s（仿真提速配置）	0.53 m/s
墙面距离	净宽一半，初值 0.85 m	参考线距可见墙 0.30 m
角速度上限	2.0 rad/s	4.3 rad/s
响应处理	曲率滤波 0.20 s；角加速度 4.0	预测 0.10 s；角速度反馈增益 0.5
关键控制量	PID 4/0/12；朝向增益 3.0	前瞻 0.32 m；返航前瞻 0.28 m
终止	出口确认 0.10 s；停稳确认 0.50 s	出口确认 0.8 s；起点容差 0.10 m

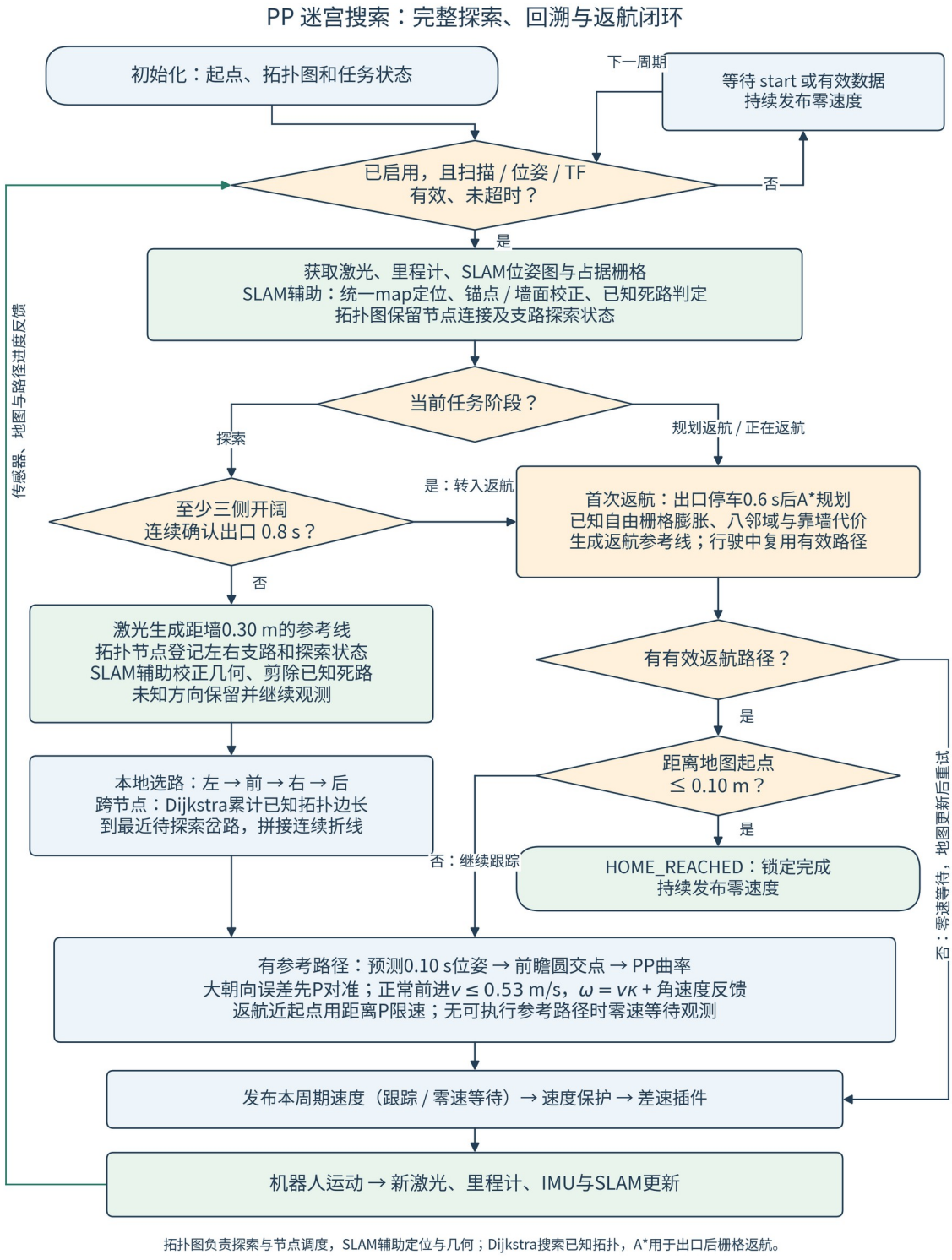


图 6 PP 探索、拓扑回溯与 A*返航闭环流程

5 测试结果与现象分析

5.1 实验条件与基础功能测试

实验平台为第 3 章所述 Gazebo 差速仿真环境。PID 合并迷宫实验于 2026 年 10 月 27 日完成，PP 的 maze2 探索与返航实验于 2026 年 10 月 20 日完成；记录内容包括位姿、速度指令、实测速度、控制状态及任务结果。2026 年 11 月 2 日完成拆包后的构建、启动和短时运动检查，原始日志、截图及测试报告随附件保存。

表 9 基础功能与任务测试

测试项目	实现与观测	结果
模型、雷达与里程计	两入口均接收 360 点扫描和 odom	传感器与运动接口正常
start / 暂停	等待阶段零指令；启用后非零；暂停后零	控制门控通过
开环运动接口	teleop_k$e_y$board.py 支持直行和原地转向	提供底盘调试入口
比例闭环与路径跟踪	朝向 P 对准、返航距离 P 及参考段跟踪	集成于 PP 任务控制流程
完整任务	PID 的 EXIT_REACHED 与 PP 的 HOME_REACHED	沿墙驶出停车、搜索后返航停车

5.2 PID 合并迷宫运行结果

早期单墙方案在 S 弯遮挡时丢失左线，引发继续寻墙和贴墙恢复。加入双侧可靠几何估计、提高角速度变化率后，1.0 m/s 轮次从正式入口完整通过，启用至确认完成约 83.52 s。之后按要求进行 1.5 m/s 试验，暴露出出口外立柱会打断较长确认窗口的问题；最终把确认从 0.20 s 缩短到 0.10 s，在保持其余速度与转向参数不变的情况下完成正常停车。

表 10 PID 合并迷宫实验轮次，统计时间为启用至确认完成

实验轮次	关键配置/现象	运行结果
full_final	1.0 m/s；出口确认 0.20 s	约 83.52 s，EXIT_REACHED，未出现恢复状态

实验轮次	关键配置/现象	运行结果
full_speed15	1.5 m/s；出口确认仍 0.20 s	约 70.10 s 报告完成，出口外出现绕柱和贴墙现象
full_speed15_exitfix	1.5 m/s；出口确认 0.10 s	约 58.08 s，正常驶出并停稳，无原地右转/贴墙恢复/丢线左转状态

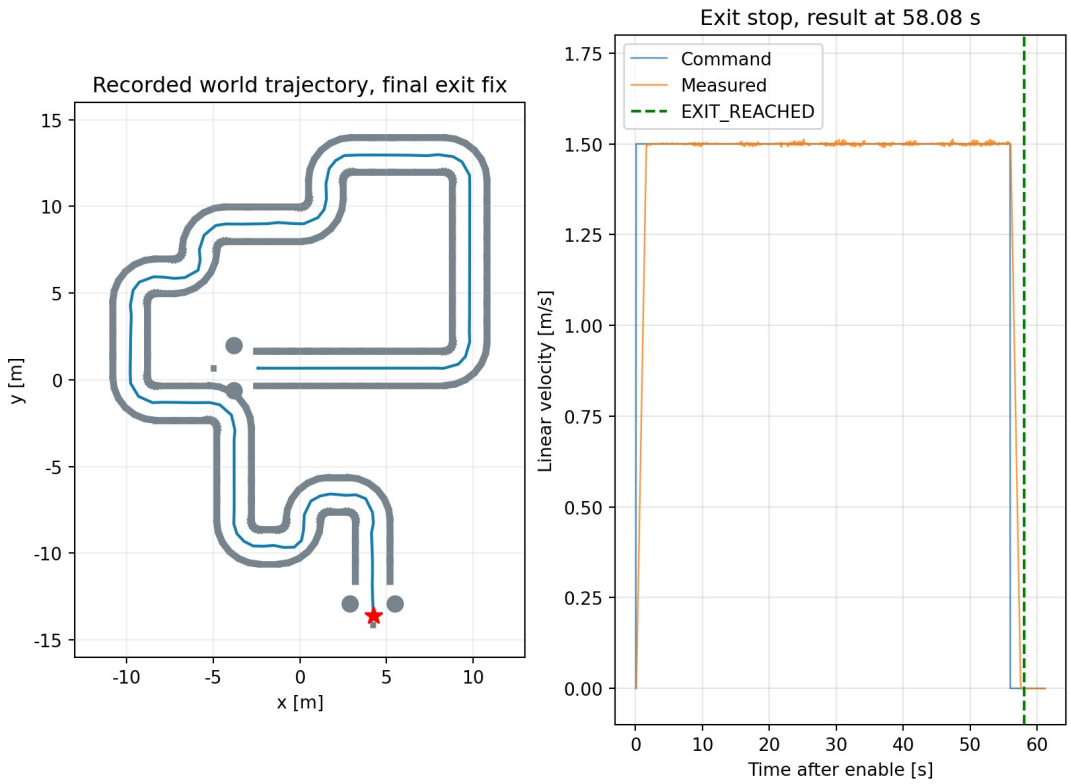


图 7 PID 最终轮次的世界坐标轨迹与速度曲线

最终轮次停车位置约为世界坐标(4.238,-13.595,0.009887) m。EXIT_BRAKING 共记录 21 帧，随后持续保持 EXIT_REACHED，速度曲线显示车辆完成制动并停稳。对比旧 1.5 m/s 轮次可见，缩短出口确认窗口后，机器人及时脱离墙端，消除了出口外绕柱和贴墙恢复现象。

在该 Gazebo 场景中，联合调整巡航速度与出口确认参数后，完成时间由约 83.52 s 缩短至 58.08 s，体现了提速配置与出口状态机配合后的整体效果。

图 8 为 2026 年 11 月 3 日使用原有启动配置补拍的任务完成画面。Gazebo 显示机器人驶出合并迷宫后的停车位置，RViz 显示里程计坐标系中的实际行驶轨迹，并以 EXIT REACHED 标记完成状态；该次运行未启用 SLAM 建图。

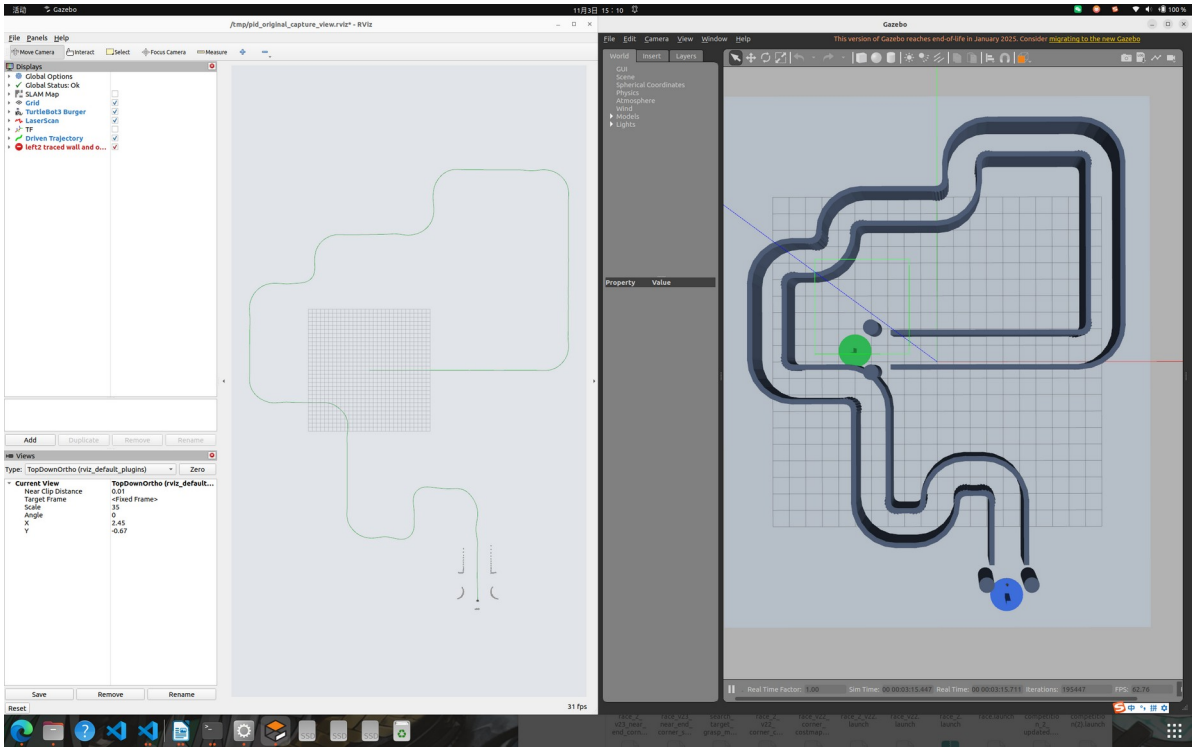


图 8 PID 驶出合并迷宫并停车的 Gazebo 与 RViz 画面，RViz 显示实际轨迹与 EXIT REACHED

5.3 PP 迷宫搜索与 A* 返航结果

maze2 实验中，机器人在线记录左右缺口与道路连接，按左优先探索，并沿 Dijkstra 规划的拓扑路线转移到待探索岔路；识别出口后计算 A*返航路径。最终进入 HOME_REACHED，完成探索、出口识别、返航规划与 PP 追踪的完整流程。

表 11 maze2 探索与返航实验结果

指标	实测记录值	含义
出生点	(-2.275,-3.8,0.01)， $yaw=\frac{\pi}{2}$	maze2 正式入口
A*路径长度	16.839 m	已知安全地图中的返航参考长度
搜索扩展单元	18 326	该轮 A*扩展量，一次规划获得路径
出口至回家	32.64 s（仿真时间）	从 PLAN_RETURN 阶段到 HOME_REACHED
起点地图距离	0.0951 m	SLAM 地图坐标下的终止距离，小于 0.10 m 容差
停车观察	约 90.74 s；非零指令样本为 0	确认完成后持续停车

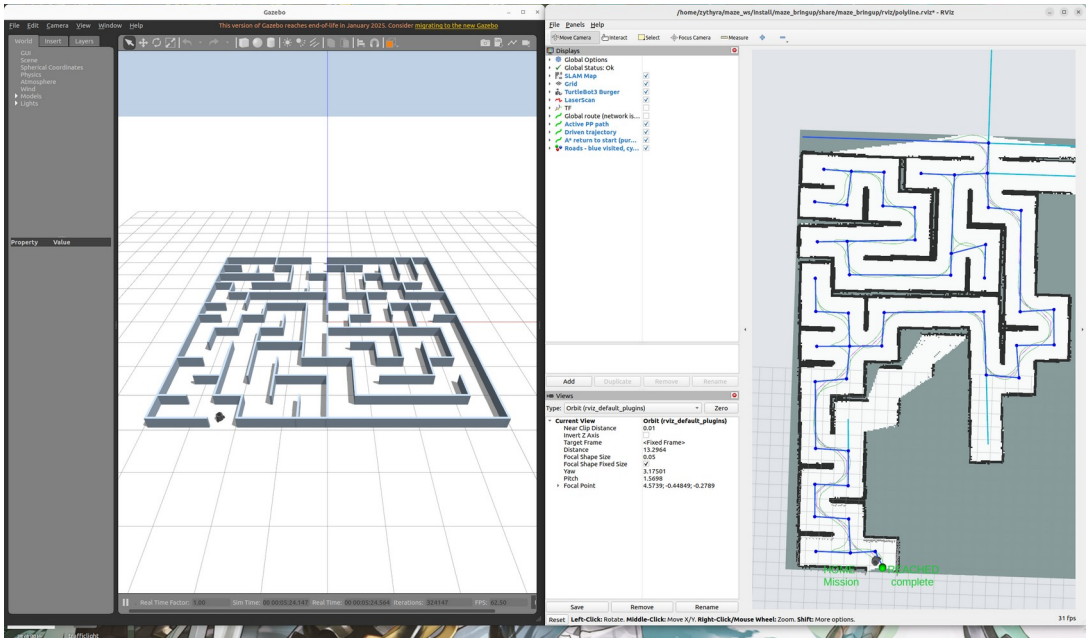


图 9 maze2 探索后返航的 Gazebo 与 RViz 画面，右下方显示 HOME_REACHED

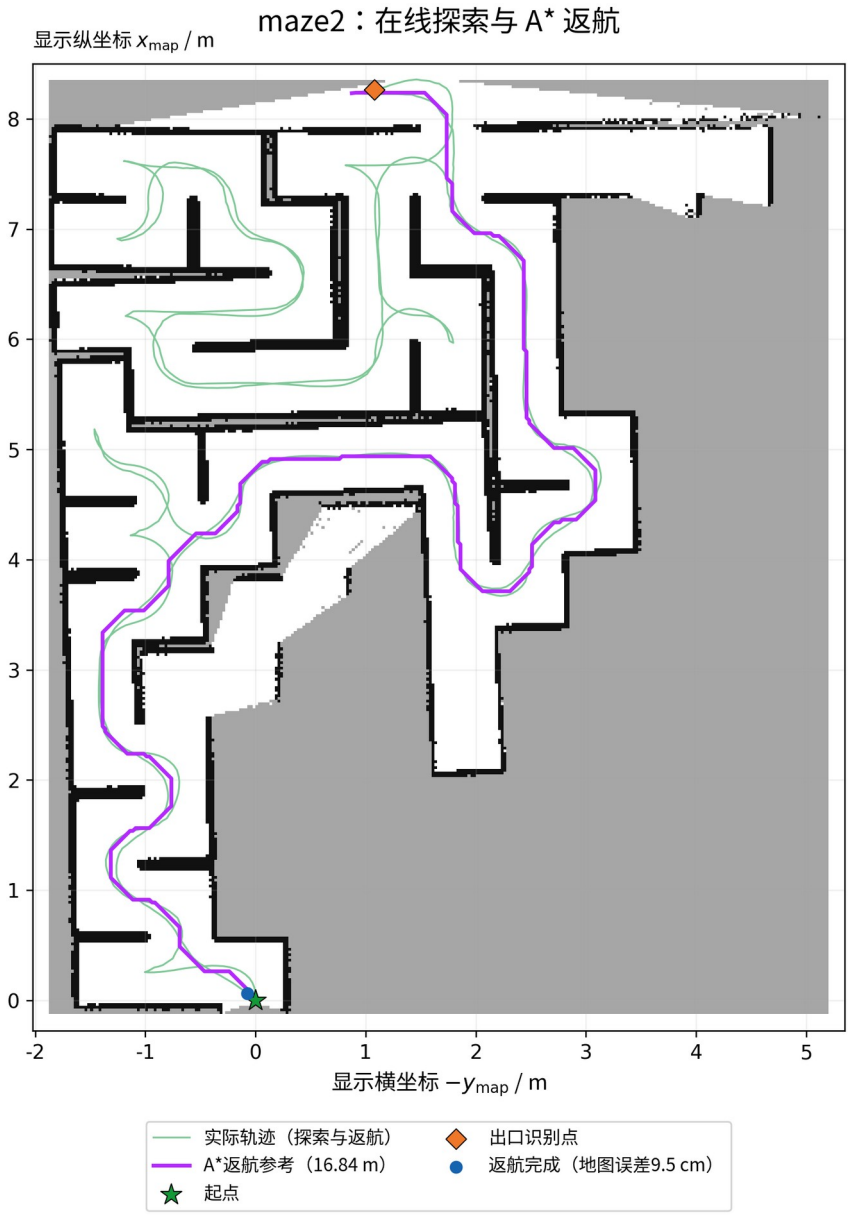


图 10 SLAM 地图与运行轨迹：绿色为实际轨迹，紫色为 A*返航参考，星形为起点

车辆返航后的地图距离为 0.0951 m，满足 0.10 m 终止容差，并在随后约 90.74 s 的观察期内持续保持零速度指令，完成起点到达与停车锁定。

5.4 回归测试

表 12 回归测试汇总（2026 年 11 月 2 日，共 125 用例）

测试文件	用例数	测试结果
test_left2_trace.gtest.xml	18	0 失败，包含双墙辅助与 PID 回归
test_polyline.xunit.xml	63	0 失败，折线与探索逻辑

测试文件	用例数	测试结果
test_return_planner.xunit.xml	13	0 失败，返航栅格规划
test_branch_map.xunit.xml	24	0 失败，地图约束与死路判定
test_terminal_start.xunit.xml	7	0 失败，终端启动门控

五组回归测试共 125 个用例，覆盖墙线几何、探索状态、返航规划、地图约束与启动门控，全部通过。单元测试用于检查计算和状态转换，Gazebo 完整路线实验用于评价运动响应及任务完成效果，二者共同构成验证流程。

6 问题分析、解决方法与改进

6.1 关键故障与处理过程

表 13 来自实际开发记录的故障、原因与修复

实际现象	原因分析	处理方法与效果
负角度方向读取错误	把-15°直接换成数组负索引或夹到 0，忽略角度布局	按角度差归一化并遍历有效扇区，支持 0～2π 与±π
转弯频繁过冲、目标线抖动	短窗口法向噪声经 0.85 m 偏移放大；持续曲率缺少前馈	扩大拟合窗口，加入曲率/朝向项及实测角速度反馈
S 弯左线突然丢失	内弯只剩短片段，切线不可信，右侧外墙仍清晰	按弧长/残差融合双墙估计，提高角速度变化率
高速出口追着外围柱绕行	开阔窗口短于确认时间，立柱使候选反复重置	正侧小扇区判别；合并图确认改 0.10 s；驶出后制动并等待实测停稳
死路逐个节点回退、重复建路	历史连接保存不足；里程/地图修正导致几何位置漂移	左右支路同时登记；Dijkstra 拼接回溯路；位姿图锚点与墙约束修正
还没扫到拐角就提前判死路	把未知或局部不可见误当已封闭	已观测区域满足闭合条件时剪枝，未知方向继续观测
RViz 视角追随或鼠标无法移动	目标帧绑定车体，或精简配置缺少交互工具	固定 Target Frame，补 MoveCamera、Views 等；保持手动操作

仿真底盘还曾出现偏航滞后与滑移。通过比较速度命令与 IMU 实测角速度，进一步定位到官方 URDF 与仿真 SDF 的惯性差异、固定坐标 link 默认质量及支撑球接触参数。simulation 准备模块在临时车模上完成相应修正。编译、资源和 GUI 问题按构建环境、安装路径、进程资源环境逐层排查，使控制算法问题与仿真运行问题得到分别处理。

6.2 方法优势、局限与改进方向

PID 方案计算量小、反馈直接，适合边界连续且通道结构明确的走廊；双墙几何辅助改善了曲线内侧遮挡时的观测。其局部控制依赖墙面可见性，缺少支路记忆。PP 方案通过拓扑图保存道路和待探索支路，支持复杂岔路中的搜索与最短路线转移；相应地，节点一致性和 SLAM 定位质量成为规划层的关键影响因素。

出口检测采用适用于课程迷宫的开阔区域判据，大型房间或装饰障碍可能造成误判。后续可增加带方向的出口门线、环境边界语义或独立标记确认，并结合实测减速度与通信时延估计停车距离。路线评价可同时统计横向误差、恢复次数、任务完成状态和总耗时。

6.3 工程权衡、质量意识与协作

提高巡航速度会压缩有效观测时间，并增加制动距离。以资源允许范围内的实时性、可诊断性和实际完成质量共同选择参数，体现了工程中的成本与性能权衡。速度保护、断流停车、启动等待和完成锁定把“何时可以动、何时必须停”显式写入程序，使安全约束成为系统接口的一部分。

团队以独立模块组织开发，统一消息、坐标和时间接口，通过状态量、轨迹曲线和原始日志交流问题。保留失败轮次与参数版本，便于复现故障、对比修复效果，也为两条方案的集成和后续维护提供依据。

7 分工、AI 使用说明与开源信息

7.1 组员分工

表 14 第 11 组成员分工

组员	独立负责模块	主要工作与交付
丁俊豪 24061435	PP 迷宫搜索项目	设计 PP 探索框架；实现折线拓扑、左优先搜索、死路回溯及 A*返航；完成调参、实验分析与对应报告
刘韩旭 24061412	PID 左墙追踪项目	设计左墙追踪方案；实现墙点提取、PID 与双墙几何辅助、出口停车；完成调参、实验分析与对应报告
两人共同	系统接口与成果整理	确定整体框架与模块接口；统一模型/话题/坐标；集成场景、讨论问题并整理报告

7.2 AI 工具使用说明

AI 工具在既定人工方案下辅助代码编写、ROS 2 接口适配、排错与日志整理；报告阶段辅助资料整理、文字润色、流程图绘制及排版。系统框架、算法方案、参数取舍和实验结论由组员确定，代码说明以现有工程及 GitHub 源码为依据，实验分析采用实际仿真记录。

7.3 开源与外部资源标注

项目仓库：https://github.com/Zythyra/maze_ws 。报告采用版本 61b83f83df0547c05d0216452d4d7ea4b6e5398f，课程代码为(2026-2027-1)-S0610350-02，学期为 2026—2027 学年第一学期。

机器人描述与仿真模型使用 ROBOTIS 官方资源，在线建图采用 slam_toolbox；PID 墙线提取和有界 PID 改编自既有 left2.cpp。maze2.world 与 merged_maze.world 由提供的场景文件接入，完成出生位置、仿真光照和启动配置适配。外部代码来源见本节和代码注释，模型许可证保留在 vendor 目录。

墙点提取与 PID 实现参考代码：https://github.com/Zythyra/ifly2_2026/blob/main/src/line_follow/src/left2.cpp 。机器人资源来源：https://github.com/ROBOTIS-GIT/turtlebot3 、https://github.com/ROBOTIS-GIT/turtlebot3_simulations 。模型说明见 ROBOTIS TurtleBot3 e-Manual，雷达角度定义依据 ROS 2 common_interfaces 中的 LaserScan.msg。

8 总结、展望与课程建议

8.1 项目总结

本项目围绕同一差速仿真对象形成局部墙追踪与带记忆迷宫搜索两种方案，完成了从模型、ROS 2 消息、激光处理到运动控制和任务结束的系统连接。PID 项目说明了传感器几何质量、反馈与转向动态共同决定曲线效果；PP 项目说明了“追踪给定路径”和“决定探索哪条路”应当分层设计。开发过程中的缺口遗漏、死路回溯和出口停车问题，也体现了状态管理与数据有效性的重要性。

8.2 后续方向

后续可在真实差速底盘上标定轮距、轮径与延迟，降低速度验证停止距离，再逐步迁移仿真控制；在现有两项目内改进视野不足时的置信度、出口门线判别和可重复的批量评估。对地图搜索，可进一步评价节点合并和回环校正的鲁棒性，比较探索路程与已知地图返航代价，减少缺口误检造成的重复行驶。

8.3 对课程实践的建议

建议课程提供统一的接口检查清单，先完成雷达角度、TF 链、仿真时间及速度停止验证，再进入综合迷宫。实验记录同时保存参数版本、失败现象和修复理由，便于小组之间复现实验并讨论设计取舍。

9 参考文献

[1] COULTER R C. Implementation of the pure pursuit path tracking algorithm: CMU-RI-TR-92-01[R/OL]. Pittsburgh: Carnegie Mellon University, 1992[2026-11-03]. https://publications.ri.cmu.edu/implementation-of-the-pure-pursuit-path-tracking-algorithm.

[2] MACENSKI S, JAMBRECIC I. SLAM Toolbox: SLAM for the dynamic world[J]. Journal of Open Source Software, 2021, 6(61): 2783. DOI:10.21105/joss.02783.

10 其他附件说明

10.1 报告与复现实验材料

实验资料源于本组工程归档：PID 原始日志见配套 evidence/merged_curve_tuning；PP 结果见 evidence/astar-return-validation.json 及对应截图。当前代码定位见本报告第 3 章和附录清单。

报告对应第 11 组：丁俊豪（24061435）、刘韩旭（24061412）。复现材料包含工程源码、地图、启动文件、参数和实验记录；源码版本与第 7 章所列 GitHub 提交一致。

表 15 报告与复现实验材料

材料	内容说明	组织方式
Word 与 PDF	综合设计报告及 PDF 电子版	包含原理、实现、实验结果与问题分析
源码	src、README、world、launch、模型、参数、测试	按 maze_wall_pid 与 maze_bringup 两个项目组织
演示视频要求	时长不超过 3 分钟	展示模型与雷达、运动过程、任务结束及算法说明
实验记录	运行 JSON、压缩日志、截图、测试报告	配套素材目录提供文件索引与 SHA256

10.2 项目启动与观测命令

两个项目的启动命令，运行时选择其中一个入口。

```
source /opt/ros/humble/setup.bash
source ~/maze_ws/install/local_setup.bash
# PID 左墙追踪
ros2 launch maze_wall_pid merged.launch.py
# PP 迷宫搜索与返航：与上一入口二选一
ros2 launch maze_bringup maze2.launch.py
```

启动后在同一终端输入 start。使用 ros2 topic echo /scan --once 和 ros2 topic echo /odom --once 观察传感器，通过各项目的 status/result 话题查看探索、回溯、返航及完成状态。